# Süreç Senkronizasyonu (Process Synchronization) - Sınav Hazırlık Özeti

## 1. Senkronizasyon Neden Gereklidir? (Sorunun Temeli)

Birden fazla prosesin (veya thread'in) aynı anda çalışabildiği (concurrent) ortamlarda, ortak verilere (shared data) aynı anda erişilip değiştirilmek istenmesi **veri tutarsızlığına (data inconsistency)** neden olur. Bunu önlemek için işletim sistemi, proseslerin çalışma sırasını **senkronize** etmelidir.

### Yarış Durumu (Race Condition) - 🚨 Banko Soru!

Aynı veriye eşzamanlı olarak erişmeye ve üzerinde işlem yapmaya çalışan birden fazla prosesin, verinin son değerini **"kimin en son bitirdiğine"** bağlı olarak değiştirmesi durumudur. Hangi prosesin / thread'in daha hızlı (veya yavaş) çalıştığına bağlı olarak verinin bozulmasıdır.

> 💡 ***Banko Sınav Sorusu Modeli:*** *Ortak bir banka hesabında iki proses aynı anda para çekmeye / yatırmaya çalışır ve bakiye yanlış hesaplanırsa bunun adı nedir?* ***Cevap:*** *Race Condition (Yarış Durumu).*

---

## 2. Kritik Bölge Problemi (The Critical-Section Problem)

Her prosesin kodunda, ortak verilerin değiştirildiği (yazıldığı) bir kısım vardır. Bu kod bölümüne **Kritik Bölge (Critical Section - CS)** denir.

> ❓ ***Hoca Sorabilir:*** *"Kritik bölge problemini çözmek için bir algoritmanın sağlaması gereken* ***3 mutlak şart*** *nedir?"*
> ✅ ***Cevap:***
>
> 1. ***Mutual Exclusion (Karşılıklı Dışlama):*** *Bir proses kendi kritik bölgesinde işlem yapıyorsa,* ***başka hiçbir proses kendi kritik bölgesine GİREMEZ.*** *(En önemli kuraldır).*
> 2. ***Progress (İlerleme):*** *Hiçbir proses kritik bölgede değilse ve bazısı girmek istiyorsa, kimin gireceği kararı sonsuza kadar ERTELENEMEZ (Mutlaka biri ilerlemelidir).*
> 3. ***Bounded Waiting (Sınırlı Bekleme):*** *Bir proses kritik bölgeye girmek için talepte bulunduktan sonra, o talebin kabul edilmesine kadar geçecek sürenin (diğer proseslerin bekleme süresinin) bir "sınırı" (limiti) olmalıdır. (Yani bir proses sonsuza dek dışlanamaz).*

---

## Peterson Çözümü - 🚨 Klasik Soru

BU KONU KESIN SINAVDA SORULUR! İki prosesin kritik bölge sorununu çözmek için `turn` ve `flag[]` değişkenleri kullanan **yazılımsal** klasik bir algoritmadır. Günümüz modern mimarileri düzensiz çalıştığı için artık tam işe yaramaz.

- `flag[i]` = Proses i'nin kritik bölgeye girmek istediği (true) veya istemediği (false) bilgisini tutar.
- `turn` = Hangi prosesin sırada olduğunu gösterir (0 veya 1).
- Proses i kritik bölgeye girmek istediğinde `flag[i] = true` yapar, diğer prosesin sırasını `turn = j` yaparak verir. Sonra da `while (flag[j] && turn == j)` döngüsüyle diğer prosesin kritik bölgeye girmesini bekler.

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);

        critical section

    flag[i] = false;

        remainder section

} while (true);
```

> ❓ ***Hoca Sorabilir:*** *"Peterson algoritması neden modern mimarilerde düzgün çalışmaz?"* ✅ ***Cevap:*** *Modern işlemciler, performansı artırmak için komutları yeniden sıralayabilir (out-of-order execution) ve bu da* `flag` *ve* `turn` *değişkenlerinin beklenmedik şekilde güncellenmesine neden olabilir. Bu nedenle, Peterson algoritması* ***doğru çalışmayabilir*** *ve bu yüzden gerçek dünyada kullanılmaz.*

## 3. Donanımsal Senkronizasyon Çözümleri

Çözümler genellikle donanım destekli "bölünemez" (Atomic) talimatlarla (komutlarla) yapılır.

- **Atomic Instruction (Atomik Komut):** Çalışırken yarıda kesilemeyen (interrupt edilemeyen), başlandığında mutlaka bitirilen tek parça komuttur.
- **Örnekler:** `TestAndSet()` ve `CompareAndSwap()` komutları. Donanım seviyesinde bu komutlar kilit (lock) görevi görerek aynı anda iki prosesin kritik bölgeye girmesini engeller.

---

## 4. Mutex Kilitleri (Mutex Locks) - Yazılımsal Çözüm

"Kritik bölgeyi korumak ve yarış durumunu önlemek için kullanılan en basit yazılımsal araçtır."

- **Mutex** = **Mut**ual **Ex**clusion (Karşılıklı Dışlama) kelimelerinin birleşimidir.
- Kritik bölgeye girmeden önce kilit alınır ( `acquire()` ), bölge bitince kilit serbest bırakılır ( `release()` ).

### Spinlock (Meşgul Bekleme / Busy Waiting)

Bir proses kilitli bir Mutex'i almak istediğinde, kilit açılana kadar sürekli olarak "Açıldı mı? Açıldı mı?" diye döngüde beklemesidir.

> ❓ ***Hoca Sorabilir (Eksisi / Artısı):*** *Spinlock'un dezavantajı nedir?* ✅ ***Cevap:*** *Sürekli çalışarak* ***CPU zamanını israf etmesi (Busy Waiting)*** *dir. Ancak avantajı şudur: Bekleme süresi çok kısaysa (Kritik bölge çok hızlı çalışıp bitiyorsa),* ***Context Switch (bağlam değişimi)*** *yapmaya gerek kalmadığı için çok hızlıdır. Çok çekirdekli (multicore) sistemlerde sık kullanılır.*

---

## 5. Semaforlar (Semaphores) - 🚨 Klasik Sınav Sorusu

Mutex'ten daha gelişmiş, tamsayı (integer) değer alan bir senkronizasyon aracıdır. Başlangıçta Hollandalı bilim insanı Dijkstra tarafından bulunmuştur. `wait()` ve `signal()` komutları (veya P ve V) ile çalışır.

1. **Binary Semaphore (İkili Semafor):** Sadece 0 ve 1 değerini alır. **Hoca Detayı:** Mutex lock'a tamamen benzer (eşdeğerdir).
2. **Counting Semaphore (Sayan Semafor):** Sınırlandırılmamış bir tamsayı alır. **Hoca Detayı:** Belirli sayıda kaynağın (örneğin 5 yazıcının) kullanımını kontrol etmek için kullanılır.

> 🚨 ***Hatırlatma:*** `wait()` *(bekle/azalt) işlemi semafor değerini eksiltir. 0 ise prosesi uyutur (bekleme kuyruğuna atar).* `signal()` *(sinyal/arttır) işlemi semafor değerini arttırır ve uyuyan bir proses varsa onu uyandırır. Bu bekleme Spinlock yapmaz, CPU israfını (Busy Wait) önler!*

---

## 6. Kritik Senkronizasyon Hataları (Deadlock ve Starvation)

Hocalar bu ikilinin farkını sormaya bayılır!

### Deadlock (Kilitlenme / Ölümcül Kilitlenme)

İki veya daha fazla prosesin, birbirlerinin elindeki kilitli kaynağı beklemesi ve **hiçbirinin sonsuza dek ilerleyememesi** durumudur.

- **Benzetme:** Dar bir köprüde karşılaşan iki inatçı keçinin "önce sen geri çık" diye birbirini saatlerce beklemesi.

### Starvation (Açlık / Süresiz Bloklama)

Bir prosesin bekleme kuyruğundan **bir türlü sırasının gelmemesi (kaynağa erişememesi)** durumudur.

- **Benzetme:** LIFO (son giren ilk çıkar) bir kuyrukta, sürekli yeni işlerin gelmesi sebebiyle ilk gelen adamın işlemlerinin "sürekli ertelenmesi" (ama teorikte bir gün çıkabilir).
- **Farkı:** Deadlock'ta herkes kilitlenir, sistem durur. Starvation'da sistem akmaya devam eder ama sadece bir/birkaç gariban proses asla kaynak bulamaz.

---

## 7. Klasik Senkronizasyon Problemleri (Banko Gelen Vakalar)

### A) Bounded-Buffer Problemi (Üretici-Tüketici / Producer-Consumer)

- **Sorun:** Sınırlı bir depo (Buffer) var. Üretici depo doluyken yeni ürün koymamalı, Tüketici de depo boşken ürün alamamalıdır. Semaforlar ile depo boş/dolu durumları eşzamanlı korunur.

### B) Readers-Writers Problemi (Okuyucular - Yazarlar)

- Veritabanı sorularıdır. (Örn: Havayolu rezervasyonu veritabanı).
- **Sorun:** Birden fazla Okuyucu aynı anda veriyi okuyabilir (Çünkü veri bozulmaz). AMA bir Yazar geldiğinde **sadece tek başına** olmalı ve yazma bitene kadar ne başka yazar ne de okuyucu sisteme girmemelidir. (Starvation çok yaşanır, genelde yazarlar aç kalır).

### C) Dining Philosophers Problemi (Filozofların Yemeği)

- Hocaların favori senaryosudur. Yuvarlak masada 5 filozof var (Prosesler), aralarında 5 çubuk (Kaynak/Semafor).
- **Sorun/Kural:** Bir filozof yemek yemek için hem solundaki hem sağındaki çubuğu almak ZORUNDADIR.
- **Kaza Senaryosu:** Herkes aynı anda solundaki çubuğu alırsa sağ çubuk için herkes kilitlenir = **İşte bu tam bir DEADLOCK (Kilitlenme) durumudur!**

---

## 8. Monitörler (Monitors)

Semaforları programcıların **yanlış kullanmasına** (Örn: `wait()` ve `signal()` sırasını karıştırması) çok sık rastlanır. Bunu çözmek için geliştirilen **üst düzey programlama dili yapısıdır (high-level abstraction).** Java, C# gibi dillerde dahili olarak bulunur (Örn: Java'da `synchronized` kelimesi). Sadece tek bir prosesin monitör içinde aktif olmasını programcı yerine dil kendisi garanti eder.

---

## 🎯 9. KISA SİYAH BİLGİ KARTLARI (HIZLI TEKRAR İÇİN)

- **Peterson Çözümü:** İki prosesin kritik bölge sorununu çözmek için `turn` ve `flag[]` değişkenleri kullanan **yazılımsal** klasik bir algoritmadır. Günümüz modern mimarileri düzensiz çalıştığı için artık tam işe yaramaz.
- **Priority Inversion (Öncelik Tersine Çevrilmesi):** Düşük öncelikli bir prosesin, yüksek öncelikli bir prosesin ihtiyaç duyduğu kaynağı (kilitleri) işgal etmesiyle oluşan, öncelik sırasını altüst eden bir hatadır. **Çözümü:** Priority-inheritance (Öncelik devri) protokolüdür (Düşük proses yüksek prosesin önceliğini geçici olarak ödünç alır).

---

## 📝 10. SINAV PROVASI: BOŞLUK DOLDURMA VE KLASİK SORULAR

1. Ortak bir veriye aynı anda erişilip sadece "proseslerin bitiş sırasına göre" verinin son değerinin belirlenmesine (yanlış yazılmasına) **[ Race Condition (Yarış Durumu) ]** denir.
2. Hiçbir prosesin sonsuza kadar dışlanmaması, illa ki belirli bir süre sonunda işlemine izin verilmesi şartına **[ Bounded Waiting (Sınırlı Bekleme) ]** denir.
3. Programcının işini kolaylaştıran, `wait` ve `signal` gibi komut karmaşasını dillerin yapısı içine (Örn: Java synchronized) yediren soyut yapıya **[ Monitor ]** denir.
4. Semaforlardan farklı olarak, sadece 0 ve 1 değeri alabilen ve kritik bölgeyi kilitlemede kullanılan basit yapıya **[ Mutex Lock / Binary Semaphore ]** denir.
5. Eğer kilit mekanizması kullanırken bekleme işlemi prosesi tam olarak uyutmayıp CPU'da "Sürekli meşgul döngüye" sokuyorsa buna **[ Spinlock (veya Busy Waiting / Meşgul Bekleme) ]** denir.
6. Masada 5 kişinin oturduğu, kilitlenme (deadlock) konseptini en iyi açıklayan senkronizasyon tasarım problemine **[ Dining-Philosophers (Filozofların Yemeği) ]** adı verilir.

### ❓ KLASİK SORU POTANSİYELİ:

**Soru:** *Kritik bölge problemine sunulacak bir algoritmanın/çözümün sağlaması gereken üç temel şartı yazınız ve Yalnızca "Mutual Exclusion (Karşılıklı Dışlama)" şartını kısaca açıklayınız.*

**Cevap:** Çözüm;

1. Mutual Exclusion (Karşılıklı Dışlama)
2. Progress (İlerleme)
3. Bounded Waiting (Sınırlı Bekleme) olmak zorundadır. **Karşılıklı dışlama (Mutual Exclusion) prensibi:** Bir an için sadece ve sadece TEK BİR prosesin o kritik koda (kod bölgesine) girmesine izin verilmesidir. Bir proses içerideyken diğerleri mutlaka dışarıda beklemelidir. Hiçbir istisnası veya eşzamanlı erişimi yoktur.